

WORKDAY ORCHESTRATE FOR INTEGRATIONS

Workday Orchestrate Complete Guide

Components explained step by step, why each is used, and hands-on builds: calling a report, reading a CSV file, and an inbound Custom ID update.

Architecture

Component deep-dive

Interactions

Security

Components

Expressions

RaaS & WQL

CSV inbound

Error handling

Deployment

Prepared by Nithin · September 23, 2026

Contents

1. Overview — what Orchestrate is and when to use it
2. Architecture and core concepts
3. Prerequisites and setup — step by step
4. Component reference — every component in detail
5. How components work together
6. Working with data — expressions, formats and parameters
7. Security and authentication
8. Sample 1 — Call a custom report (RaaS) and produce an output file
9. Sample 2 — Read and validate a CSV file
10. Sample 3 — Inbound: update worker Custom IDs from an input file
11. Deploy, launch, monitor and troubleshoot
12. Best practices, build checklist and sources

1. Overview — what Orchestrate is and when to use it

Workday Orchestrate is a low-code, drag-and-drop builder for integration workflows (orchestrations) that run inside the Workday platform. You build them in the Orchestration Builder on the Workday Developer Site, package them in an app, promote the app to a tenant, and link the orchestration to a Workday integration system that launches it.

Orchestrations run either as event-driven workflows (triggered by an action such as a hire or a payroll change) or as scheduled/launched batch processes. They add decision logic on top of classic integration systems: conditions, loops, error handlers, logging, and calls to RaaS, WQL, Workday SOAP/REST and external APIs.

In plain words: think of Orchestrate as a recipe card. The **App** is the recipe book, each **Orchestration** is one recipe, each **step** is one instruction ("fetch the report", "for each row...", "call Workday"), and the **Integration System** is the kitchen timer in your tenant that says "cook it now". The **integration event** is the notebook where the cook writes what happened.

Two Orchestrate products

Product	Used for	Components only it has
Orchestrate for Integrations	Integration systems launched from Workday (inbound/outbound, scheduled)	Send Workday RaaS Request, Trigger Business Process, Trigger Integration, Trigger PDF Generation, Store Documents, Log to File, Add Integration Message
Orchestrate for Extend	Orchestrations called from Extend apps (PMD/AMD), sync and async	AWS components (Extend Professional)

This guide covers **Orchestrate for Integrations**, which is what you need for report-driven outbound feeds and file-driven inbound loads.

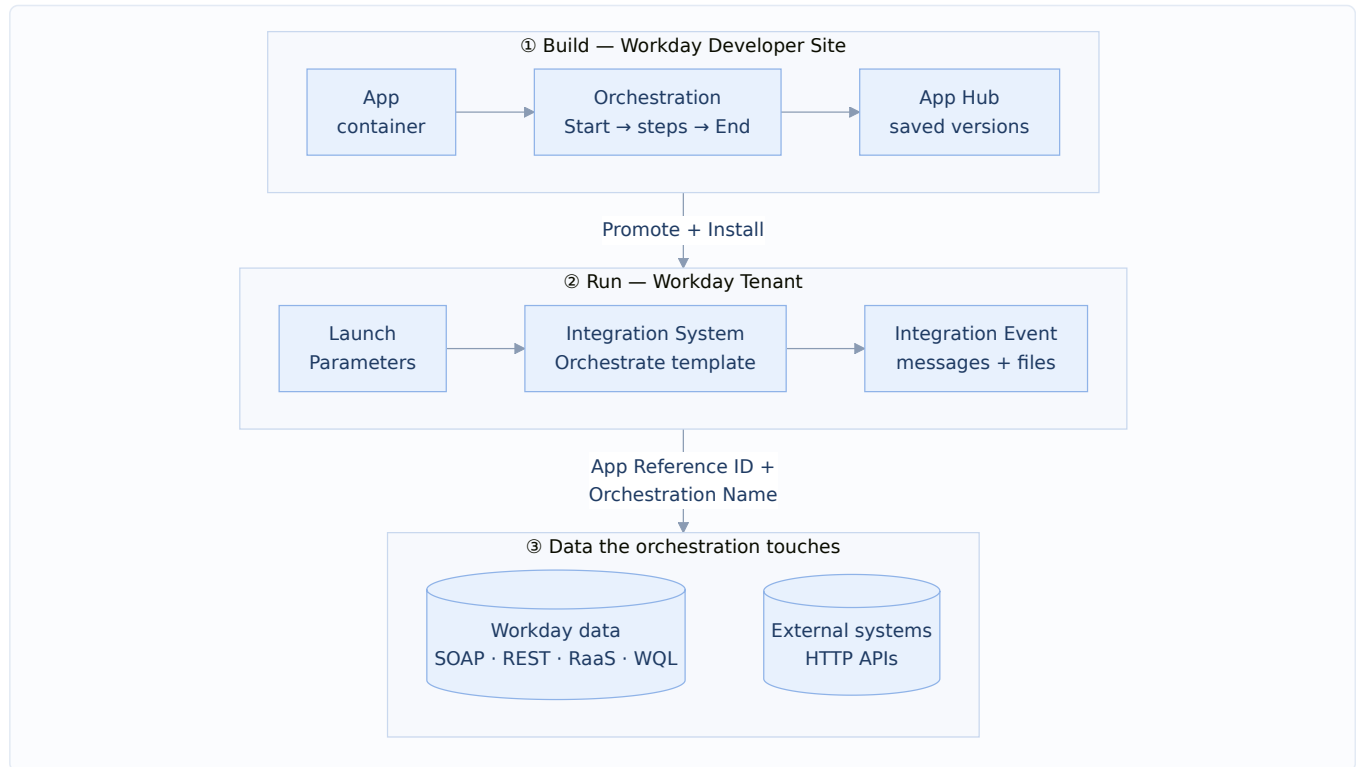
Choosing the tool

Tool	Pick it when	Avoid it when
Orchestrate for Integrations	Transactional or moderate-volume flows with logic, API calls, validation, messaging	Very large batch jobs (hundreds of thousands of rows)
Workday Studio	Complex, specialised logic, heavy XSLT/java, very large files	Simple flows — longer build and steeper learning curve
Core Connector / EIB	Standard data out or in with no transformation	You need branching, lookups or API calls (send connector output to an orchestration instead)

Rule of thumb from Workday Education: treat Orchestrate as the default integration tool, use connectors only where no transformation is needed, and keep Studio for the specialised minority of requirements.

2. Architecture and core concepts

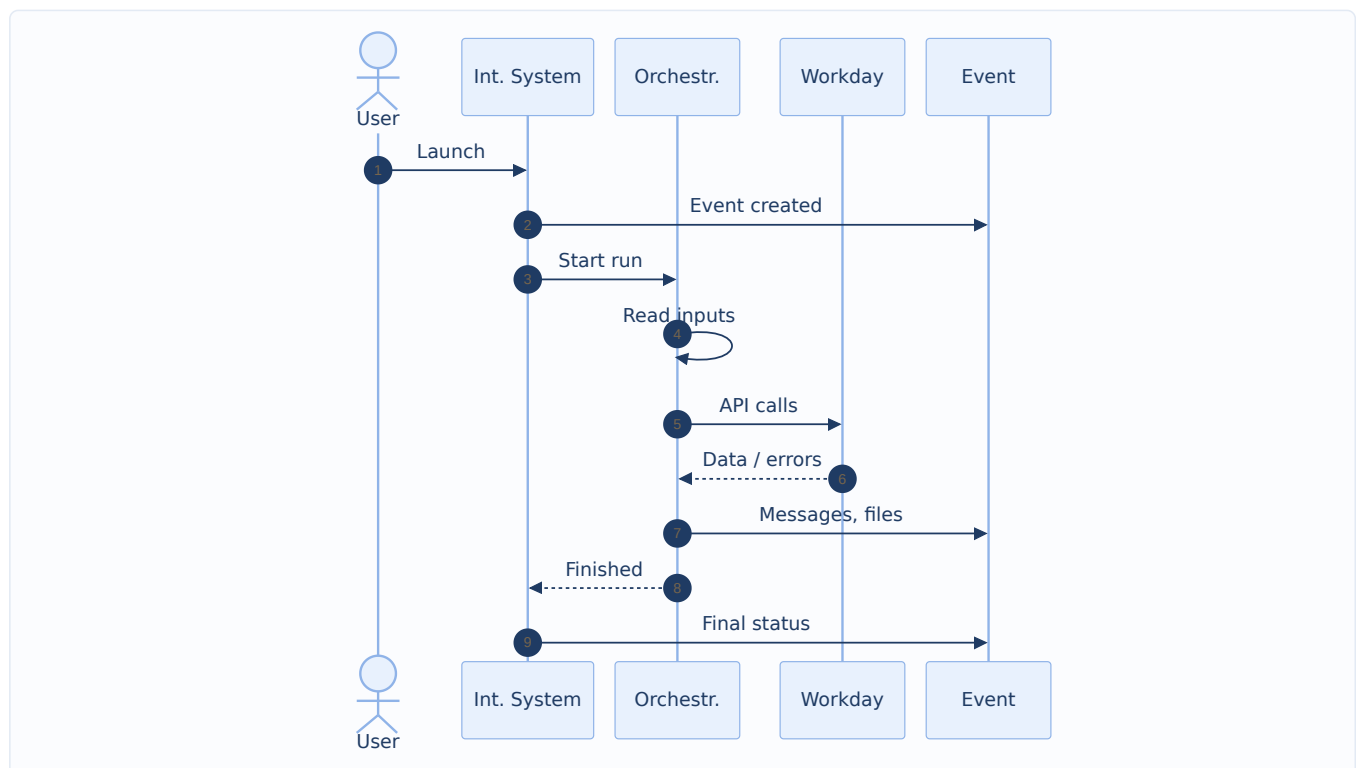
Every Orchestrate integration has three parts: an **App** on the Developer Site, one or more **Orchestrations** inside it, and an **Integration System** in the tenant that launches the orchestration. The app and orchestration live in the App Hub; the integration system lives in the tenant and points at them through two attributes.



Read it left to right: you build in the Developer Site, promote to the tenant, and at runtime the integration system calls the orchestration, which reads and writes Workday data and external systems and reports back to the integration event.

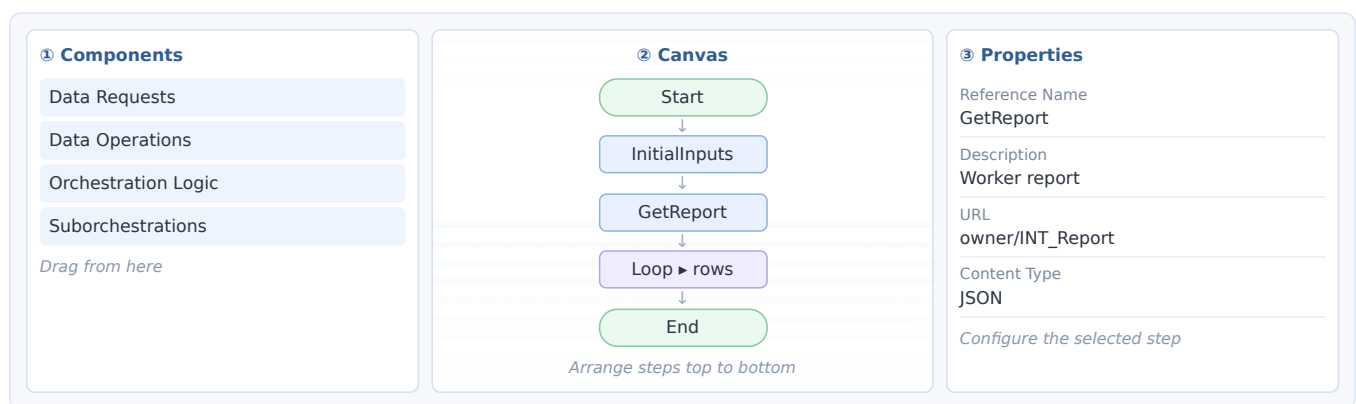
Life of a single run

This is what happens, in order, when someone clicks *Launch* on the integration system.



Every sample later in this guide follows this same shape. Only the steps in the middle (5–7) change.

The Orchestration Builder screen



You drag a component (①) onto the canvas (②), where it becomes a step. Clicking the step opens its properties (③), where you fill values with the Expression Builder. The toolbar also holds **Validate**, **Save All to App Hub**, the **global error handler** icon, and the **Build Logs / Run Logs** panel at the bottom.

Key terms

Term	What it is	Why it matters
App	Container for related orchestrations; can hold shared app attributes	The unit you save, version, promote and install
Orchestration	The flow: Start step → your steps → End step	Holds all logic; one app can hold many
Component vs step	A component in the left panel becomes a <i>step</i> once dropped on the canvas	Each step has a Reference Name you use to read its output later
Canvas / Components / Properties	The three views of the Orchestration Builder	Drag from Components, arrange on Canvas, configure in Properties
Integration System	Tenant object built with the Orchestrate Integration Template	Launches the orchestration, carries launch parameters, owns the integration event
Integration attributes	Orchestration Name + Application Reference ID on the integration system	The link between tenant and app — a typo here means the run fails
Launch parameters	Runtime inputs defined on the integration system (text, boolean, number, date, instance)	Change behaviour per run without redeploying (dates, IDs, validate-only mode)
Expression	Value of a property built in Pill Mode or Advanced Mode	How every step reads upstream data, launch params and global functions
Short ID	Your company's suffix appended to every app name	Keeps your apps separate from other companies' in the App Hub

Rules that shape every design

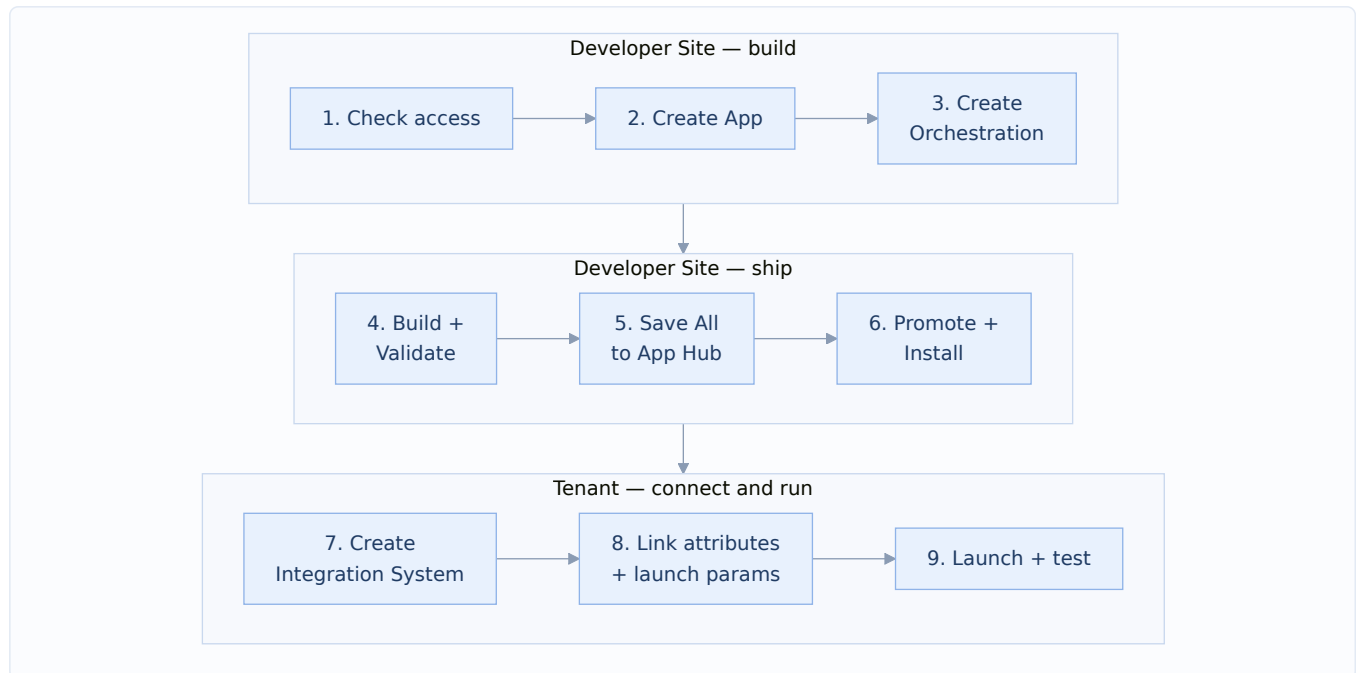
- **Data flows downward only.** A step can read outputs of steps above it, never below. Order your steps so every value exists before it is used.
- **Reference names are your variables.** Rename every step (for example `SendRaaS_Workers` not `SendWorkdayRaaSRequest`) because expressions refer to them by name.
- **Integration event is your log.** Add Integration Message and Store Document write to it; the Log step writes only to Developer Site logs.
- **Promotion is one-way.** IMPL → SBX → PROD; you cannot demote, and an installed app cannot be removed from IMPL/SBX/PROD tenants.

Component categories

Components come in four families: **Data Requests** (talk to Workday or other systems), **Data Operations** (shape and save data), **Orchestration Logic** (loops, branches, messages, error handling) and **Suborchestrations** (reuse). Section 4 walks through each of these components in turn.

3. Prerequisites and setup — step by step

You need an Orchestrate licence, a Developer Site role of Developer or higher, and a tenant (IMPL or SBX) connected to your company. Follow these steps once per integration.



Step 1 — Confirm Developer Site access

1. Sign in at developer.workday.com and check the company name in the top bar next to your profile.
2. Confirm your role: Company Administrator, Developer (build + promote), Specialist (no promotion to PROD) or Member (view only).
3. Consultants need Collaborator access, requested by the customer through Customer Care > Account > Extend Collaborator Access.

Why: the company decides which tenants and apps you see; the wrong company means your app is created in the wrong App Hub.

Step 2 — Create the App

1. Console > Apps > **Create** > Integration App (or the *Create an Integration App* link on the home page).
2. Choose **Start from scratch** (or copy from App Catalog / upload a ZIP).
3. Enter a name such as `workerIntegrations` and a description. The Short ID suffix is added automatically.

Why: the app is the container you version and promote. Group orchestrations by purpose (for example all worker-data loads) so they move together.

Step 3 — Create the Orchestration

1. In the app, choose **Create Orchestration**.
2. Type: **Workday Integration System** (recommended) or Workday Business Process.
3. Name it, for example `UpdateWorkerCustomID`. The Builder opens with Start and End steps.

Why integration system over business process: you get an integration event with messages, files and launch parameters, which makes troubleshooting far easier. Use the BP trigger only if you must react to rescind, cancel or correction.

Step 4 — Build and validate

1. Drag components between Start and End; order them so data flows downward.
2. Rename each step and fill its Description.
3. Configure required properties (marked *) with expressions.

4. Add step-level and global error handling.
5. Click **Validate**. Errors must be fixed; warnings should be.

Step 5 — Save All to App Hub

Click **Save All to App Hub**. After the first promotion, each save asks whether to create a new version. If save fails with no validation error, open the **Build Logs** — the error may be in another orchestration in the same app.

Step 6 — Promote and install

1. App > **Promotions** tab > **Promote**, enter release notes, confirm. Repeat until you reach the level you test at (IMPL or SBX).
2. Click **Install** → App Manager in the tenant > *Ready to Install* tab > Install > confirm. The app moves to *Installed*.
3. Install is needed only the first time; later promotions replace the installed version.

Why: the tenant runs only what is installed. Promotion moves only the app — the integration system, custom reports and security move with OX 2.0 or manual config.

Step 7 — Create the Integration System in the tenant

1. Task **Create Integration System**.
2. Template: **Orchestrate Integration Template**.
3. Name it, for example `INT_ORCH_Update_Worker_CustomID`.

Step 8 — Link it to the orchestration

1. Developer Site > App Overview: copy **Application Reference ID** and **Orchestration Name**.
2. Tenant: integration system > Related Actions > Integration System > **Configure Integration Attributes** > + > paste both values > Save.
3. Related Actions > **Maintain Launch Parameters** for any runtime inputs (dates, IDs, *Validation Mode* boolean). Labels must match exactly what the orchestration reads.
4. Security: add the integration system to an Integration System Security Group (ISSG) with the domains it needs — for example Get on *Worker Data: Public Worker Reports* and Put on the custom ID domain — then **Activate Pending Security Policy Changes**.

Step 9 — Launch and test

Run **Launch / Schedule Integration**, then open the integration event: check Messages for your Add Integration Message entries and Output Files for stored documents. A mismatched attribute or uninstalled app shows as an error here.

Customers with only the Orchestrate for Integrations SKU have no DEV tenants; you test in IMPL or SBX.

4. Component reference — every component in detail

A **component** is a building block in the left panel; once you drop it on the canvas it becomes a **step**. Each card below follows the same pattern so you can scan quickly:

What goes in **The step** **What comes out** — plus *when to use it, key properties, an example, and watch-outs*.

Map of all components

Data Requests

talk to systems

Send Workday API Request
Send Paged Workday SOAP Call
Send Paged Workday REST Call
Send Workday RaaS Request
Send HTTP / Paged HTTP Request

Data Operations

shape and save data

Create Values
Create Text Template
Create JSON
Store Document
Log · Log to File

Orchestration Logic

control the flow

Branch · Continue on Conditions
Loop · Batch Loop · Join Loop
Add Integration Message
Error handlers
Trigger Integration / BP / PDF

Suborchestrations

reuse

Call a child orchestration with inputs and outputs

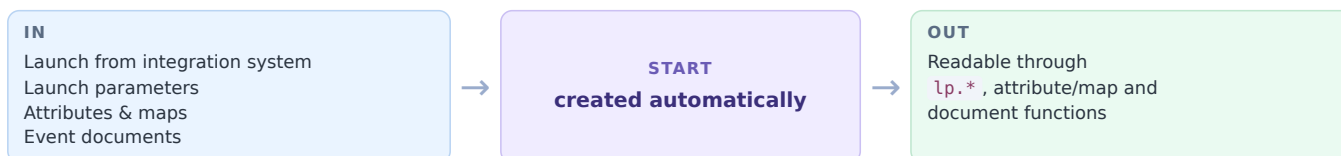
Summary table

Component	Category	In one line	Main output you use later
Start (Integration Framework Trigger)	—	Receives the launch	Launch context (params, attributes, files)
Create Values	Data Operations	Named variables	<code>data.<Step>.<key></code>
Create Text Template	Data Operations	Text with placeholders	The rendered text
Create JSON	Data Operations	Typed JSON object	JSON value
Send Workday API Request	Data Requests	One SOAP or REST call to Workday	<code>response</code> , status
Send Paged Workday SOAP Call	Data Requests	Many pages of a SOAP Get_	Iterator of XML pages
Send Paged Workday REST Call	Data Requests	Many pages of a REST collection	Iterator / flattened JSON
Send Workday RaaS Request	Data Requests	Runs a custom report	Report as JSON / CSV / XML
Send HTTP Request	Data Requests	Call a non-Workday API	Response body, status, headers
Send Paged HTTP Request	Data Requests	Many pages of an external API	Iterator of pages
Store Document	Data Operations	Save a file (optionally on the event)	Document reference
Log	Data Operations	Developer log line	Run Logs entry
Log to File	Data Operations	CSV log file	File on the event
Add Integration Message	Orchestration Logic	Message on the event	Event message
Branch on Conditions	Orchestration Logic	IF / ELSE paths	Optional common outputs
Continue on Conditions	Orchestration Logic	Guard: go on or stop	—
Loop	Orchestration Logic	Repeat for each item	<code>Loop.item</code> , aggregations
Batch Loop	Orchestration Logic	Repeat for each group of items	Batch items, aggregations
Join Loop	Orchestration Logic	Match two datasets by key	Joined record per item
Error handlers	Orchestration Logic	Catch failures	<code>isSuccess</code> , error details
Trigger Integration / BP / PDF	Integrations only	Start other Workday work	Event / BP reference

Component	Category	In one line	Main output you use later
Suborchestration	Reuse	Call a child flow	Child's outputs

4.1 Start step (Integration Framework Trigger)

In plain words: the front door. Workday opens it when the integration system is launched and hands over everything the run needs.



When to use: always present; you cannot delete it. **Key point:** it has no properties to fill in. Put a Create Values step right after it to collect the inputs once. **Watch out:** if the integration system's *Orchestration Name* or *Application Reference ID* is wrong, the run never reaches this step.

4.2 Create Values

In plain words: a labelled box of variables. You give each value a name once and every later step just asks for it by name.



When to use: at the top of every orchestration (inputs), inside loops (one row's fields), and anywhere you would otherwise repeat the same expression.

Property	What to enter	Example
Reference Name	Purpose of the variables	InitialInputs , Row
Key	Variable name (no spaces)	fromDate
Value	Expression	lp.dateParam("From Date")
Type	AutoType or a fixed type	Date

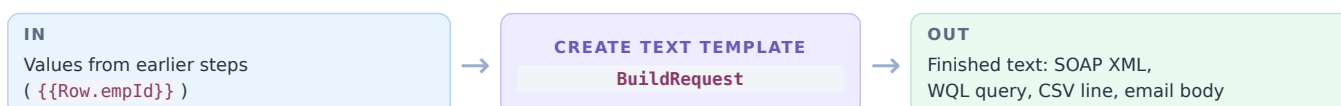
Example — one set of inputs with fallbacks:

Key	Value expression	Plain meaning
fromDate	if lp.existsAndHasValue("From Date") then lp.dateParam("From Date") else today - 7 days	Use the date the user typed; otherwise last week
validateOnly	lp.booleanParam("Validation Mode")	Dry run or real run
idType	stringAttributeValueOrDefault(...) with default BADGE_ID	Configurable per tenant

Watch out: a key can only use steps *above* it. If a value is missing from the dropdown, the source step is below or has no output.

4.3 Create Text Template

In plain words: a fill-in-the-blanks letter. You write the text once with `{{placeholders}}` and Orchestrate fills them in at runtime.



When to use: SOAP request bodies, WQL queries, simple CSV/text lines. **Example (WQL):**

```

SELECT employeeID, worker, hireDate
FROM allWorkers
WHERE hireDate >= '{{InitialInputs.fromDate}}'
  
```

Watch out: text is not validated — a missing `>` in XML only shows up when Workday rejects the call. Test the body in the SOAP API Explorer first. For JSON prefer Create JSON.

4.4 Create JSON

In plain words: a form for building JSON. You add fields one by one and Orchestrate guarantees the result is valid JSON.



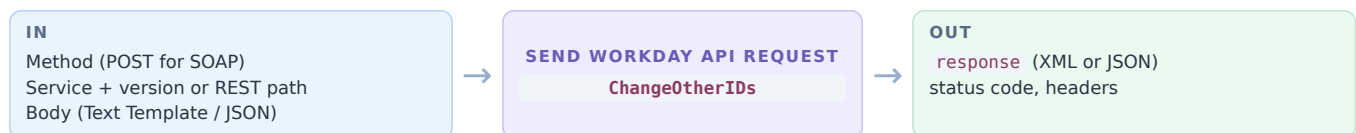
When to use: REST request bodies and when turning XML (connector/SOAP output) into JSON. **Example output for one worker:**

```
{ "employeeId": "21001", "name": "Logan McNeil", "department": "Finance" }
```

Put Create JSON inside a Loop and add a **JSON Fold** aggregation to get an array of all workers. **Watch out:** use `...WithDefault` functions for optional fields so one missing value does not fail the row.

4.5 Send Workday API Request (SOAP and REST)

In plain words: the phone line to Workday. It sends one request (read or update) and brings back Workday's answer.



When to use: reading (Get_Workers, WQL `/data`) or writing (Change_Other_IDs, Maintain_Contact_Information, REST POST/PATCH).

Property	Meaning
Method	POST for SOAP; GET/POST/PUT/PATCH/DELETE for REST
URL / service	Prefilled Workday API gateway address + service and version (SOAP) or resource path (REST)
Authentication	Default: <i>Access Token From Initiating User</i> — no API client or manual OAuth header needed
Body	Reference existing data or a Text Template
Settings	Timeouts and HTTP retry policy

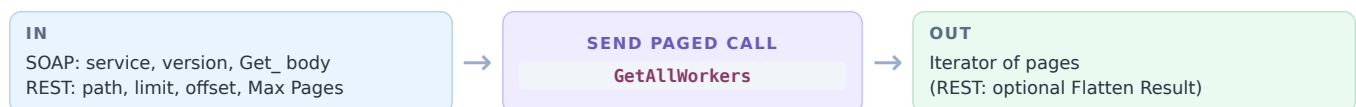
Example — read a value from a SOAP response:

```
data.GetWorker.response.unwrapSoap().stringAtXPath("/wd:Get_Workers_Response/wd:Response_Data/wd:Worker/wd:Worker_Descriptor")
```

`unwrapSoap()` removes the envelope so your XPath starts at the service response. **Watch out:** `stringAtXPath` fails when the node is missing — use `stringAtXPathWithDefault` for optional data. SOAP needs *Integration* permissions on the domain; REST needs *Report/Task* permissions (View for GET, Modify for POST/PUT/PATCH/DELETE).

4.6 Send Paged Workday SOAP Call and Send Paged Workday REST Call

In plain words: the same phone call, repeated automatically until Workday says "no more pages".



When to use: any Get_ or REST collection that can return more than one page (all workers, all positions). **How it pages:** SOAP follows `Next_Page_Reference`; REST sends repeated GETs with limit/offset until empty or Max Pages. **Example flow:** Paged SOAP Call → Loop over pages → inner Loop over `wd:Worker` nodes → aggregate. **Watch out:** do not wrap the envelope twice — if the step wraps it for you, supply only the request body. Set Max Iterations / Max Pages so a mistake cannot run forever.

4.7 Send Workday RaaS Request

In plain words: "run this report and give me the results". Your report does the filtering and calculations; the orchestration just consumes the rows.

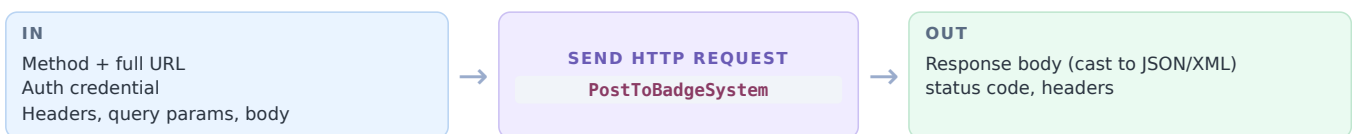


Property	Tip
URL	Only edit the pill: report owner + report name
Content Type	JSON for looping; CSV when the file goes straight out
Query Parameters	Prompt names exactly as in the report's web service view
Settings	Longer Connect Timeout for big reports; retry policy

RaaS or WQL? RaaS = reuse report logic and calculated fields, many output formats. WQL = faster, only the fields you ask for, JSON only. **Watch out:** report must be *web service enabled* and shared with the ISU/ISSG, or you get empty data or 403.

4.8 Send HTTP Request and Send Paged HTTP Request

In plain words: the phone line to everyone who is *not* Workday — a vendor API, a badge system, an exchange-rate service.



Auth options (defined in orchestration Settings): API Key, Basic, OAuth, Integration System User. **Example:** POST each worker's new badge ID (from Create JSON) to `https://badges.example.com/api/v1/assign` with header `Content-Type: application/json`. **Watch out:** never use it for Workday APIs — use Send Workday API Request, which handles Workday authentication for you. Keep base URLs in attributes so each tenant can point to its own test endpoint.

4.9 Store Document

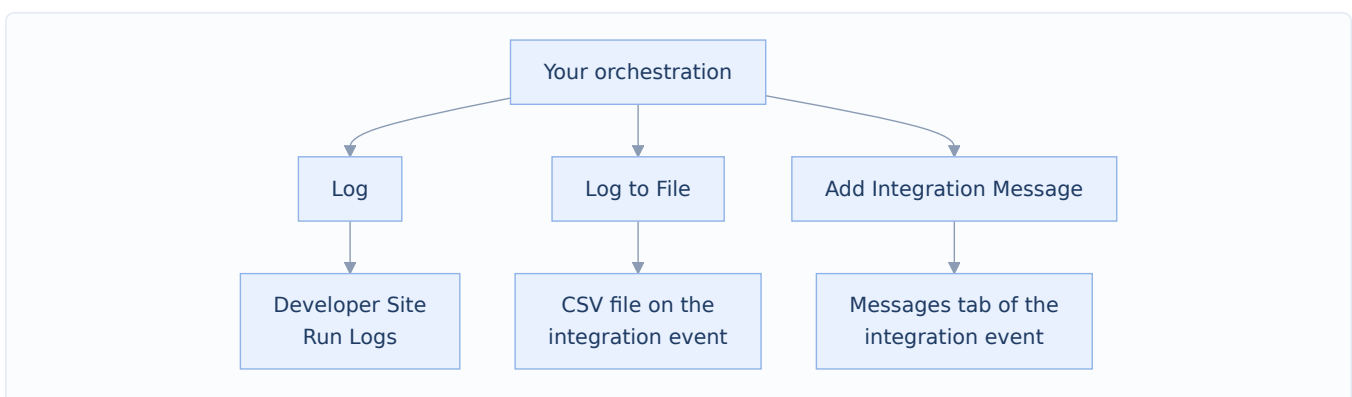
In plain words: "save this as a file". Users can download it from the integration event.



Example: Data = `LoopRows.Results` (a CSV aggregation), name `CustomID_Results.csv`, attach = on. **Watch out:** attach to integration event is **off by default**; document tags must already exist in the tenant or the step fails.

4.10 Log, Log to File and Add Integration Message

In plain words: three ways to leave notes. One for you (Log), one as a spreadsheet (Log to File), one for the business user (Add Integration Message).



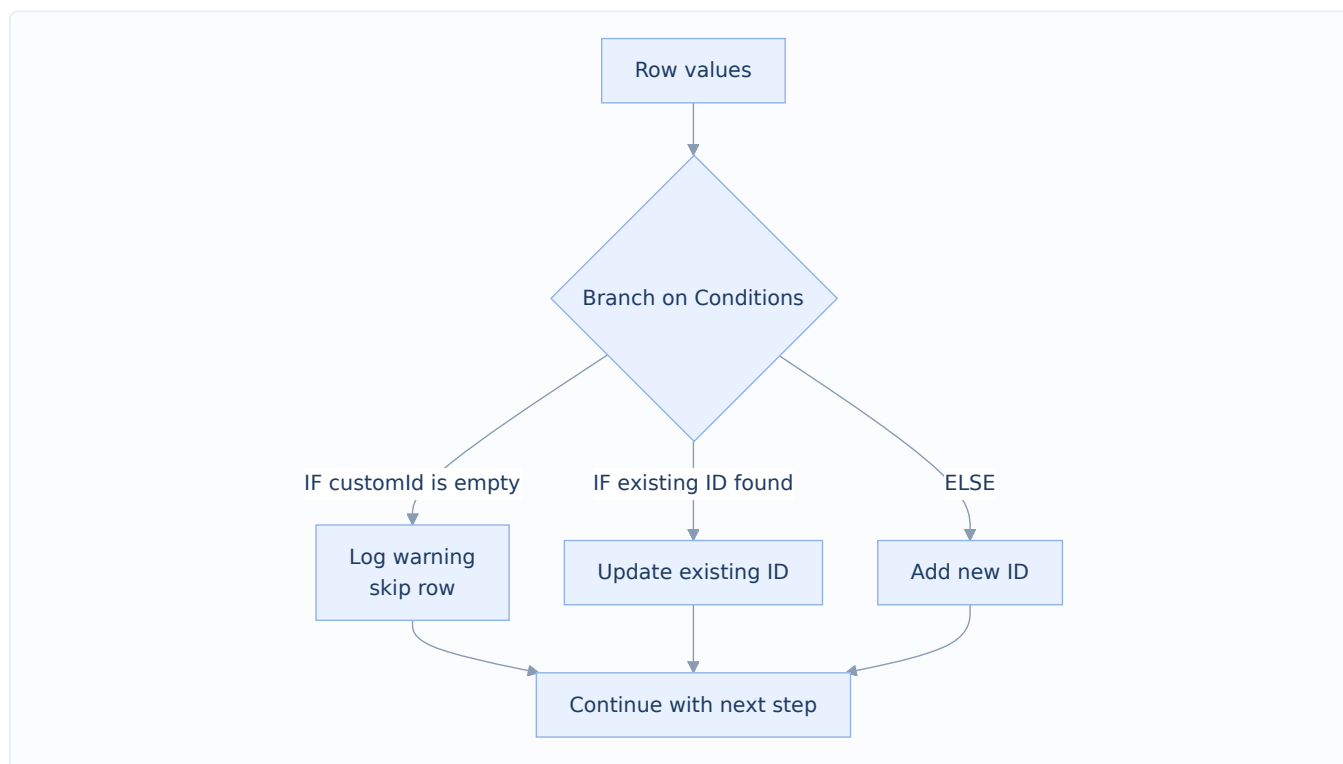
Property (Add Integration Message)	Meaning	Example
Severity	INFO, WARNING, ERROR, CRITICAL, DEBUG	WARNING
Summary	Short text shown in the list	Row skipped: 21003 missing Custom ID

Property (Add Integration Message)	Meaning	Example
Detail	Longer text	Error message from Workday
Condition	Only write when true	<code>FailedCount > 0</code>

Watch out: a Summary built with a complex expression sometimes renders poorly — use Log for the detail and a short Summary.

4.11 Branch on Conditions

In plain words: a fork in the road. Orchestrate checks the branches from left to right and takes the first one whose condition is true.



Key properties: one condition per branch (value — operator — value; combine with *All of these* = AND or *One of these* = OR), and *Enable outputs* if later steps need a value from whichever branch ran (every branch must then return the same output names and types). **Watch out:** prefer extra IF branches over deeply nested conditions — easier to read and maintain.

4.12 Continue on Conditions

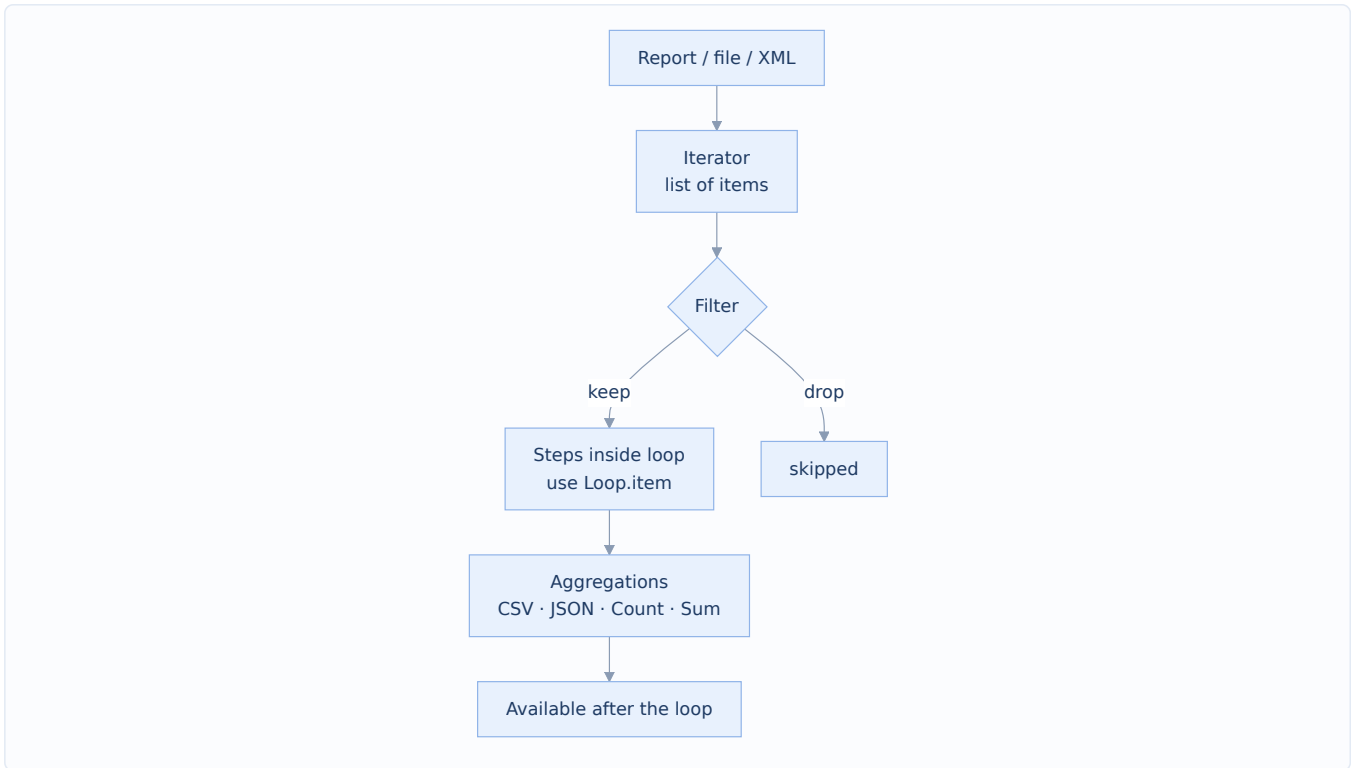
In plain words: a security guard. If the check passes, the run continues; if it fails, the run stops with your message.



Example: `lp.existsAndHasValue("Effective Date")` AND input file is defined → otherwise stop with *Effective Date and input file are required*. **Branch vs Continue:** use Branch when both outcomes are valid paths; use Continue when a false result means "do not go any further".

4.13 Loop

In plain words: "for each row, do these steps". The steps you place inside the loop run once per item.



Property	Meaning	Example
Data Type	Iterator type (JSON, XML) or AutoType	JsonIteratorType
Data Set	Expression that returns the items	<code>data.GetWorkersReport.response.iterator("\$.Report_Entry[*"])</code>
Filter	Only items where this is true enter	<code>Employee_ID</code> not empty
Sort By / Locale	Order of processing	by Hire Date
Aggregations	Combine results after the loop	CSV <code>Results</code> , Count <code>RowCount</code>

Inside the loop read fields with `data.Loop.item.stringAtJsonPath("$.Employee_ID")` (JSON) or `stringAtXPath("/Employee/ID")` (XML). **Watch out:** by default one failing iteration stops the whole loop — add an *Error Handler per Iteration* (Conditional Outputs) when partial success is acceptable.

4.14 Aggregations (on Loop, Batch Loop, Join Loop)

In plain words: a collection bucket that fills up as the loop runs and is handed over, full, when the loop ends.

Strategy	What you configure	Result	Example
Text	Text per item, separator, header, footer	One string	List of failed IDs, comma-separated
JSON / JSON Fold	JSON fragment per item	JSON array	Array of workers for a REST POST
CSV	Format (header, separator) + column mapping	CSV document	Results file
Count	Optional condition	Number	<code>UpdatedCount</code>
Sum	Number per item	Number	Total hours

Every strategy can have a **Condition**, so one loop can build a success file and an error file at the same time.

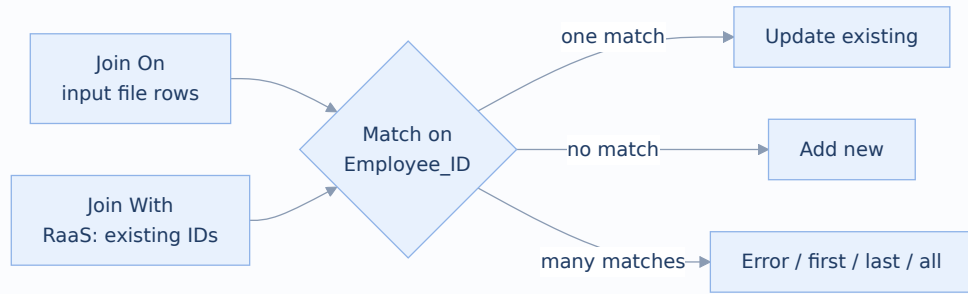
4.15 Batch Loop

In plain words: like Loop, but it hands you items in groups (for example 100 at a time) instead of one by one.

When to use: the target API accepts many records per call, or you want fewer, larger requests. **Example:** 1,000 rows, batch size 200 → 5 iterations, each building one request with 200 records.

4.16 Join Loop

In plain words: a VLOOKUP. For each record in list A it finds the matching record(s) in list B using a shared key.

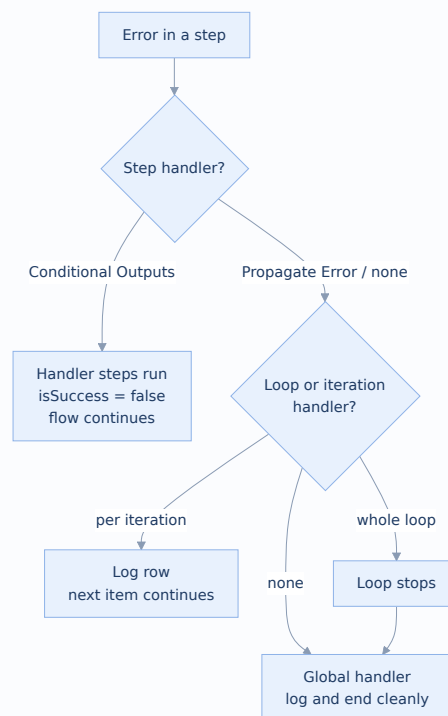


Setting	Options
Multiple matches	Complete with Error, Skip Record, Use First, Use Last, Use All
Missing match	Alternate Processing, Complete with Error, Skip Record, Add Log

Use `isDataDefined` inside the loop to test whether a Join With value exists.

4.17 Error handlers

In plain words: safety nets at different heights. The lowest net catches one step; the highest (global) catches anything nobody else caught.



Handler	Catches	Typical content	Continue after?
Step-level	One step or group	Log, retry, set fallback	Yes with Conditional Outputs
Per iteration	One loop item	Add Integration Message with the row ID	Yes — next item runs
Loop	Any item in the loop	Log which item broke everything	No — loop stops
Global	Anything not caught	<code>ProcessingError.message</code> , <code>locationId</code> , <code>locationPath</code>	No — run ends

Default for step handlers is **Propagate Error**; choose **Conditional Outputs** explicitly when you want to carry on.

4.18 Trigger Integration, Trigger Business Process, Trigger PDF Generation

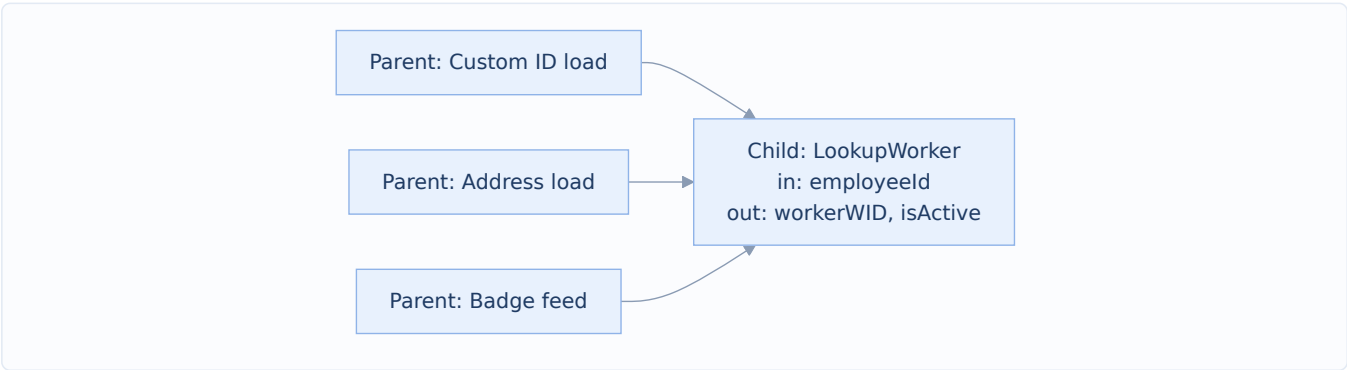
In plain words: "and when you're done, start that next thing".

Component	Example
Trigger Integration	After loading Custom IDs, launch the outbound badge-system integration
Trigger Business Process	Start a BP for each validated record
Trigger PDF Generation	Create a PDF letter or summary from a template

These are available in Orchestrate for Integrations only.

4.19 Suborchestration

In plain words: a reusable mini-recipe. Build it once ("look up a worker", "log an error the standard way") and call it from many orchestrations.

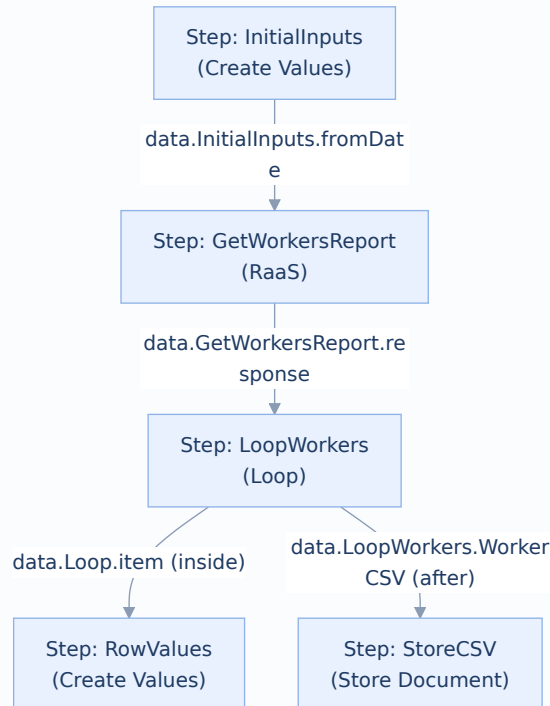


How: create the child in the same app, define its inputs and outputs, drag it into the parent and map values in, read its outputs afterwards. **Why:** one fix updates every caller; parents stay short and readable.

5. How components work together

Steps talk to each other in one way only: **a later step reads the output of an earlier step by its Reference Name**. Understanding this one rule explains almost every orchestration.

The data-passing model



In plain words: every step puts its result in a labelled drawer named after the step. Later steps open the drawer by name. That is why renaming steps clearly matters — `data.GetWorkersReport.response` tells a reader exactly where the data came from; `data.SendWorkdayRaaSRequest2.response` does not.

Three rules follow from this:

Rule	What it means	Example
Top-down only	A step sees drawers of steps above it, never below	RowValues can read InitialInputs, not StoreCSV
Inside vs after a loop	Inside the loop you see the current item; after the loop you see only aggregations	<code>data.Loop.item</code> inside; <code>data.LoopWorkers.RowCount</code> after
Branch outputs	Outputs from inside a branch are visible later only if <i>Enable outputs</i> is on and every branch returns them	<code>status</code> returned by every branch

A traced example — watching one row travel

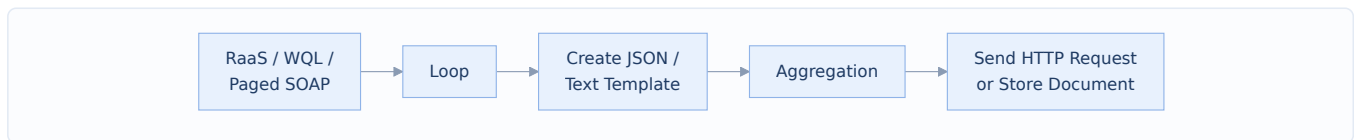
Input file row: `21001,BADGE_ID,BADGE-0001,2026-10-01` with *Validation Mode* = *false*.

#	Step	Reads	Produces (actual value)
1	<code>InitialInputs</code> (Create Values)	Launch params	<code>validateOnly = false</code> , <code>fileLabel = "INPUT"</code>
2	<code>GetInputFile</code> (Create Values)	Event documents labelled <code>INPUT</code>	<code>inputText = "Employee_ID,...\n21001,BADGE_ID,..."</code>
3	<code>CheckFile</code> (Continue on Conditions)	<code>inputText</code>	true → continue
4	<code>ParseRows</code> (Create Values)	<code>inputText</code>	<code>rows = 4 lines</code>
5	<code>LoopRows</code> (Loop)	<code>rows</code> , filter drops header	Item 1 = <code>21001,BADGE_ID,BADGE-0001,2026-10-01</code>
6	<code>Row</code> (Create Values, in loop)	<code>Loop.item</code>	<code>empId=21001</code> , <code>idType=BADGE_ID</code> , <code>customId=BADGE-0001</code>

#	Step	Reads	Produces (actual value)
7	<code>ValidateRow</code> (Branch)	<code>Row.*</code>	Valid branch
8	<code>CheckMode</code> (Branch)	<code>InitialInputs.validateOnly</code>	ELSE branch (real update)
9	<code>BuildChangeOtherIDs</code> (Text Template)	<code>Row.*</code>	SOAP XML with <code>21001</code> and <code>BADGE-0001</code>
10	<code>ChangeOtherIDs</code> (API Request)	Template text	Response 200, <code>isSuccess = true</code>
11	Aggregations	Row + status	Results CSV line <code>21001,BADGE_ID,BADGE-0001,Updated, UpdatedCount +1</code>
12	<code>StoreResults</code> + <code>Summary</code> (after loop)	<code>LoopRows.Results</code> , counts	File on event · message <i>Updated 2, invalid 1, failed 0</i>

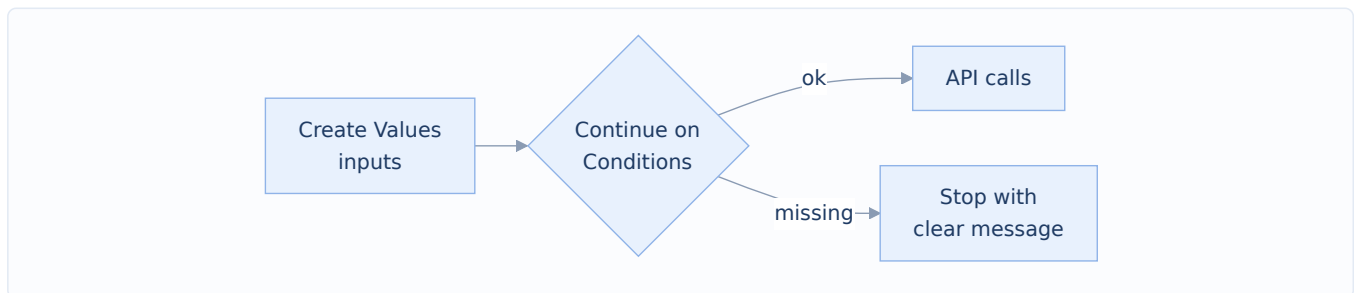
Five interaction patterns you will reuse

Pattern A — Read → Transform → Write (outbound)



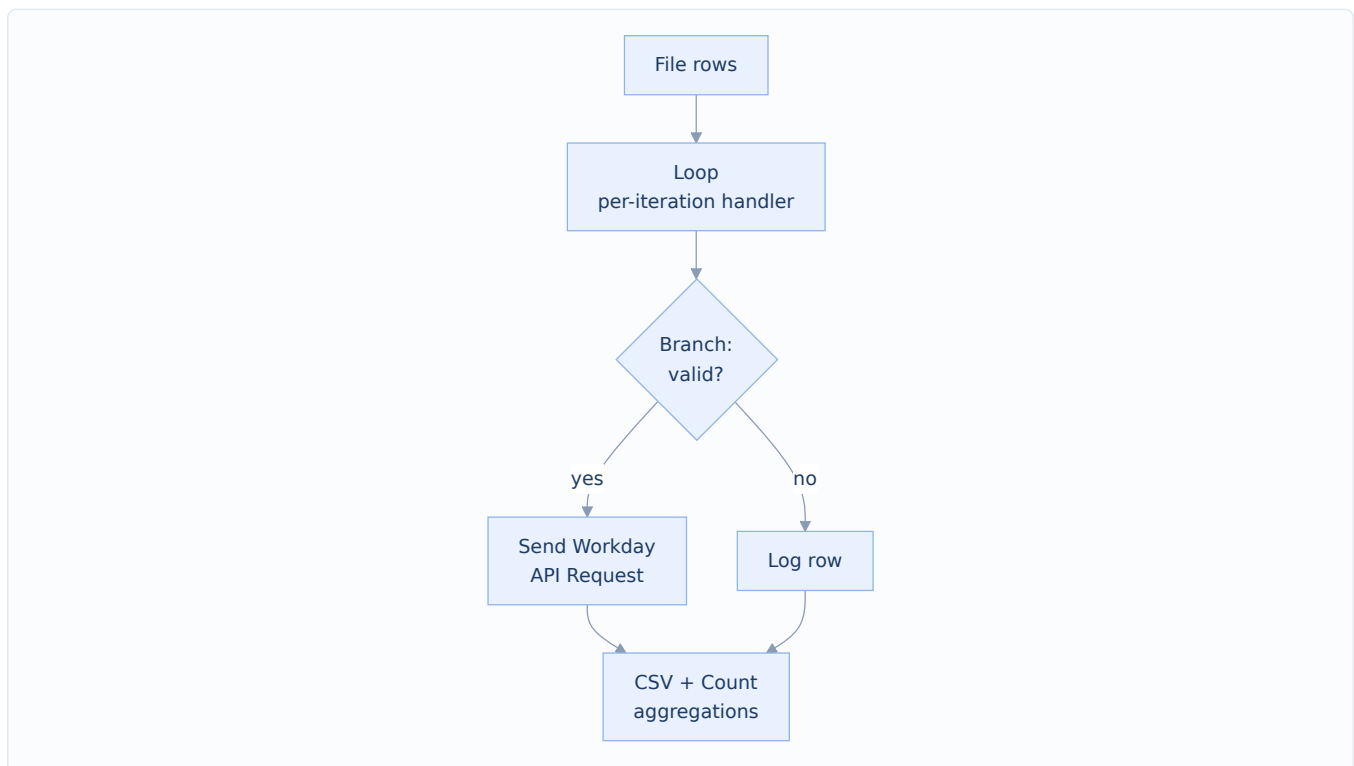
Used for: feeds to vendors, extracts to SFTP, JSON pushes to external APIs.

Pattern B — Guard → Act (fail fast)



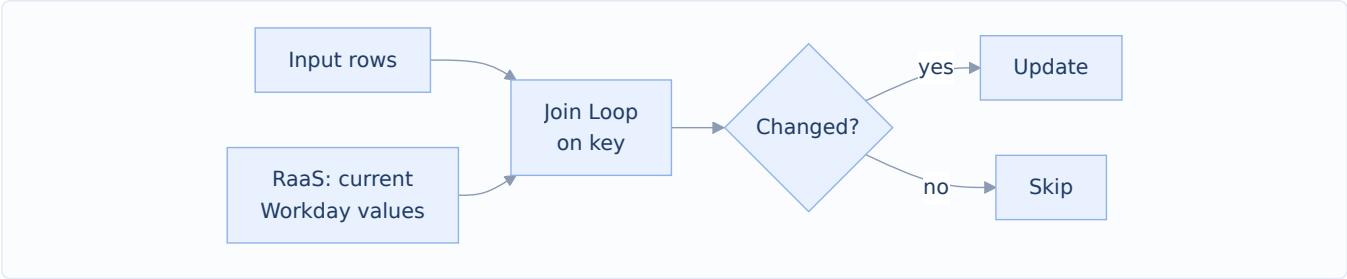
Used for: every orchestration with required launch parameters or files.

Pattern C — Loop → Validate → Act → Record (inbound)



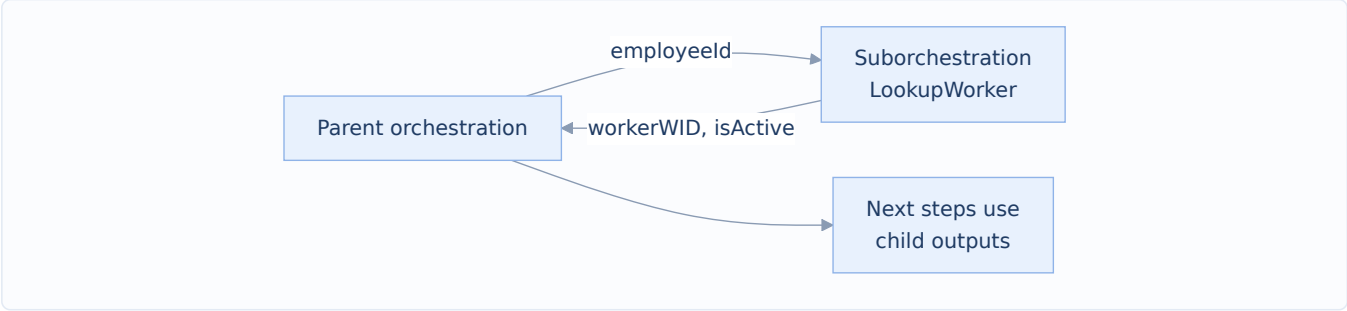
Used for: any load into Workday (IDs, contact info, compensation, custom objects).

Pattern D — Fetch → Join → Enrich (boomerang)



Used for: only updating what changed, avoiding duplicates, add-vs-update decisions.

Pattern E — Parent → Child (reuse)

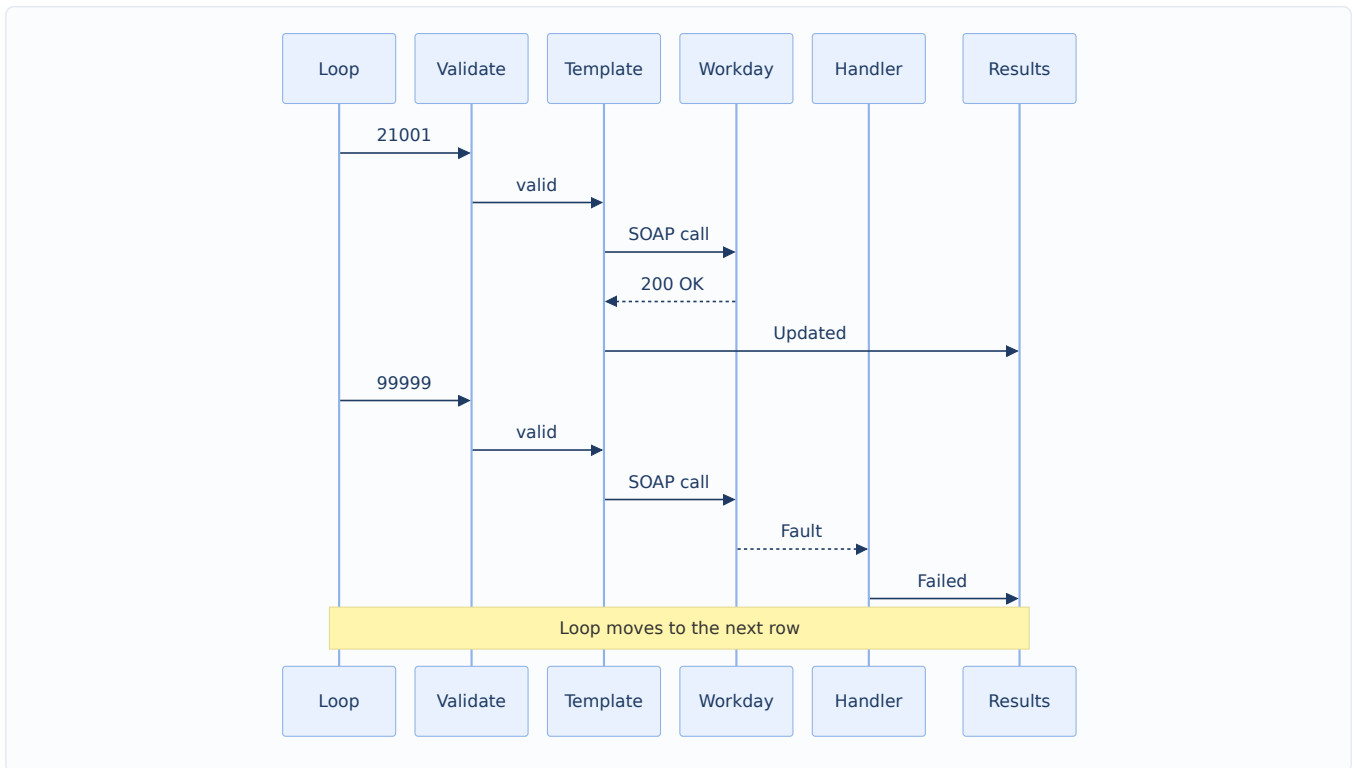


Used for: shared lookups, standard error logging, token retrieval.

Which component usually feeds which

From ↓ / To →	Create Values	Text Template / JSON	API / RaaS / HTTP	Loop	Branch / Continue	Store / Messages
Start (launch context)	✓ inputs				✓ guard	
Create Values	✓	✓ placeholders	✓ URL / params	✓ data set	✓ conditions	✓ text
Text Template / JSON			✓ body			✓ file content
API / RaaS / HTTP	✓ extract fields			✓ iterator	✓ status checks	✓ raw response
Loop (inside)	✓ row fields	✓ per row	✓ per row	✓ nested	✓ per row	✓ per-row messages
Loop (after)	✓ counts	✓ payload	✓ folded array		✓ totals	✓ files, summary

An inbound row, end to end



The per-iteration handler turns a Workday fault into a *Failed* row in the results file instead of stopping the run.

6. Working with data — expressions, formats and parameters

In plain words: an expression is a small formula, like a spreadsheet formula, that tells a property where its value comes from.

Every property value is an expression. Master four things — the Expression Builder, launch parameter functions, JSONPath/XPath, and aggregations — and you can build almost any flow.

Expression Builder

Mode / feature	What it does	Tip
Pill Mode (default)	Visual pills: text, numbers (green), booleans, step data, functions	Start here; hover a pill for <i>insert left</i> and <i>function</i> icons
Advanced Mode	Code-style editing of the same expression	Switch for long or nested expressions
Add condition for value	IF/ELSE value inside one property	Keep nesting shallow — prefer Branch steps
Menu: Data from Orchestration Steps	Outputs of earlier steps only	Missing? The source step is below or has no output
Menu: Global Functions / Explore All Functions	Function library with inputs and return types	Use Function Explorer to check exact names
Menu: Attribute Store Values	App attributes	Only shows if the app has attributes
Duplicate	Copy an expression	Saves time on many similar mappings

Launch parameter functions

Function	Returns	Example use
<code>lp.stringParam("Country Code")</code>	String	Filter value
<code>lp.simpleData("Message")</code>	Raw string	No type validation needed
<code>lp.booleanParam("Validation Mode")</code>	Boolean	Validate-only run
<code>lp.numberParam("Batch Size")</code>	Number	Batch size
<code>lp.dateParam("Effective Date")</code>	Date	Effective date of the change
<code>lp.dateTimeParam("As Of")</code>	ZonedDateTime	Incremental extracts
<code>lp.referenceData("Organization", "Organization_Reference_ID")</code>	Reference ID	Instance prompt
<code>lp.existsAndHasValue("Effective Date")</code>	Boolean	Guard in Continue on Conditions

Labels must match the integration system exactly, including spaces and case.

Expression cookbook

I want to...	Expression pattern
Read a launch parameter date	<code>lp.dateParam("Effective Date")</code>
Check a parameter was entered	<code>lp.existsAndHasValue("Effective Date")</code>
Read a JSON field in a loop	<code>data.Loop.item.stringAtJsonPath("\$.Employee_ID")</code>
Read a JSON number	<code>data.Loop.item.numberAtJsonPath("\$.Hours")</code>
Optional JSON field	<code>data.Loop.item.stringAtJsonPathWithDefault("\$.Custom_ID", "")</code>
Loop over RaaS rows	<code>data.GetWorkersReport.response.iterator("\$.Report_Entry[*]")</code>
Loop over WQL rows	<code>data.RunWQL.response.iterator("\$.data[*]")</code>
Remove SOAP envelope	<code>data.GetWorker.response.unwrapSoap()</code>
Read a SOAP value	<code>...unwrapSoap().stringAtXPath("/wd:Get_Workers_Response/.../wd:Worker_Descriptor")</code>
Count SOAP nodes	<code>...unwrapSoap().countItemsAtXPath("/wd:Get_Workers_Response/wd:Response_Data/wd:Worker")</code>
Loop over XML nodes	<code>data.CCWFile.iterator("/Employees/Employee")</code>

I want to...	Expression pattern
Test a Join With value exists	<code>isDataDefined(...)</code>
Combine checks	<code>and(condition1, condition2)</code>
Map lookup with default	<code>mapLookupWithDefault(...)</code>

Exact names and arguments can differ by release — confirm in **Explore All Functions**, which shows each function's inputs, output type and description.

Data types at a glance

Type	Looks like	Colour in Pill Mode	Comes from
String	<code>"21001"</code>	default	Most <code>stringAt...</code> functions, text
Number	<code>42</code>	green	<code>numberAt...</code> , Count/Sum aggregations
Boolean	<code>true</code>	green	<code>lp.booleanParam</code> , conditions
Date / DateTime	<code>2026-10-01</code>	—	<code>lp.dateParam</code> , date functions
JSON / XML	document	—	API responses, Create JSON
Iterator	list of items	—	<code>.iterator(...)</code> , paged calls

Reading JSON (RaaS, WQL, REST)

A RaaS JSON response looks like this — the rows are in the `Report_Entry` array:

```
{ } response
|
| Report_Entry [ ] ← loop over this
|
| [0] { Employee_ID, Worker, Custom_ID } ← Loop.item
|
| [1] { Employee_ID, Worker, Custom_ID }
|
| Employee_ID = "21001" ← stringAtPath("$.Employee_ID")
|
{
  "Report_Entry": [
    { "Employee_ID": "21001", "Worker": "Logan McNeil", "Custom_ID": "BADGE-0001" },
    { "Employee_ID": "21002", "Worker": "Steve Morgan", "Custom_ID": "" }
  ]
}
```

Need	JSONPath
All rows (loop data set)	<code>\$.Report_Entry[*]</code>
Third row (zero-based)	<code>\$.Report_Entry[2]</code>
One field inside the loop	<code>\$.Employee_ID</code> on <code>Loop.item</code>
WQL rows	<code>\$.data[*]</code>

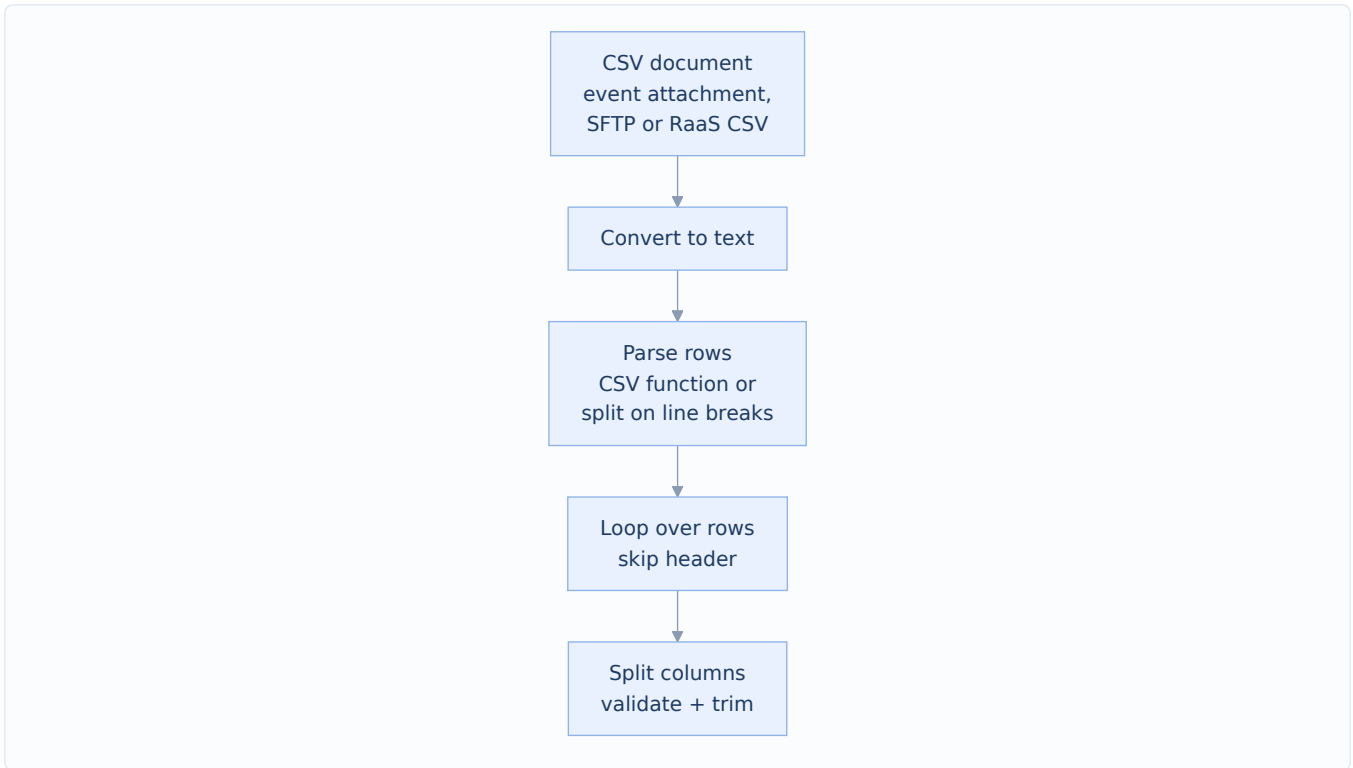
Test paths on [jsonpath.com](#) or [jsonpathfinder.com](#) with **dummy data only** — never paste PII.

Reading XML (SOAP responses, connector output)

- Create an iterator: `xmlDoc.iterator("/Employees/Employee")`.
- Inside the loop: `Loop.item.stringAtPath("/Employee/Name")`, or `stringAtPathWithDefault(..., "Unknown")` for optional fields.
- SOAP responses carry the `wd:` namespace; confirm the exact XPath with a sample response from the SOAP API Explorer.

Reading CSV

CSV has no native path language, so the pattern is: get the file as data → turn it into rows → loop.



Use the Function Explorer and search **csv** to find the delivered CSV parsing/iterator functions for your tenant version; if none fits, split the text on line breaks and then on the delimiter. Always skip the header row, trim values, and handle quoted commas.

Aggregations

Strategy	Produces	Typical use
Text	One string with separator/header/footer	Log text, delimited list
JSON / JSON Fold	JSON array	Payload for a REST call
CSV	CSV document from a format definition	Output or error file
Count	Number	Processed / failed counts
Sum	Number	Totals for control checks

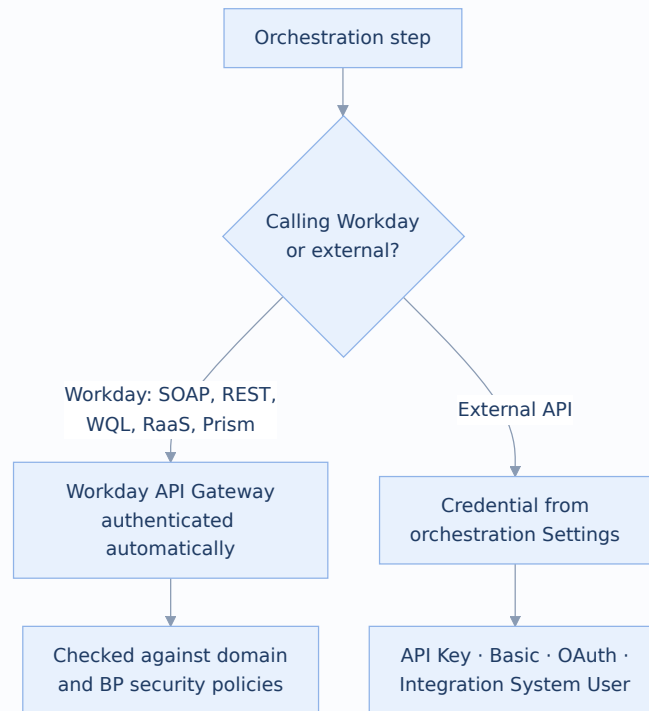
Add one via the loop's step menu > Add Aggregation; set a Condition to include only certain rows (for example `isSuccess = false` for an error file).

Integration maps and attributes

Integration maps (on the integration system) translate internal Workday values to external codes and back; read them with map lookup functions such as `mapLookupWithDefault`. Attributes hold configuration such as endpoint URLs or the Custom ID Type — read with attribute functions such as `stringAttributeValueOrDefault`. Both change per tenant without redeploying the app.

7. Security and authentication

In plain words: calls to Workday use Workday's normal security — whoever runs the integration needs the same domain access a person would need. Calls to other systems need their own keys or tokens, stored in the orchestration settings.



Workday calls

Item	What to set up	Why
Endpoint authentication	Default <i>Access Token From Initiating User</i> on Workday steps	No API client or manual OAuth header is needed
ISSG / security group	Integration system's security group added to the right domains	Workday checks every call against domain security
SOAP permissions	Integration permission (Get/Put) on the domain	SOAP only works with integration permissions
REST permissions	Report/Task permission: View for GET, Modify for POST/PUT/PATCH/DELETE	REST uses report/task permissions only
RaaS	Report enabled as web service and shared with the group/ISU	Otherwise empty data or 403
Business processes	Initiate rights on the BP policy (for example an ID change)	Put operations that start a BP fail without it
Activation	<i>Activate Pending Security Policy Changes</i>	Nothing applies until activated

Find the domain for any operation with **View Security for Securable Item** or the web service's documentation.

External calls

Credential type	Use when	Stored in
API Key	Vendor gives a key for a header or query parameter	Orchestration Settings > credentials
Basic Authentication	Username + password endpoints	Orchestration Settings
OAuth	Token-based APIs	Token or credential store reference
Integration System User	Simplified ISU authentication	Orchestration Settings

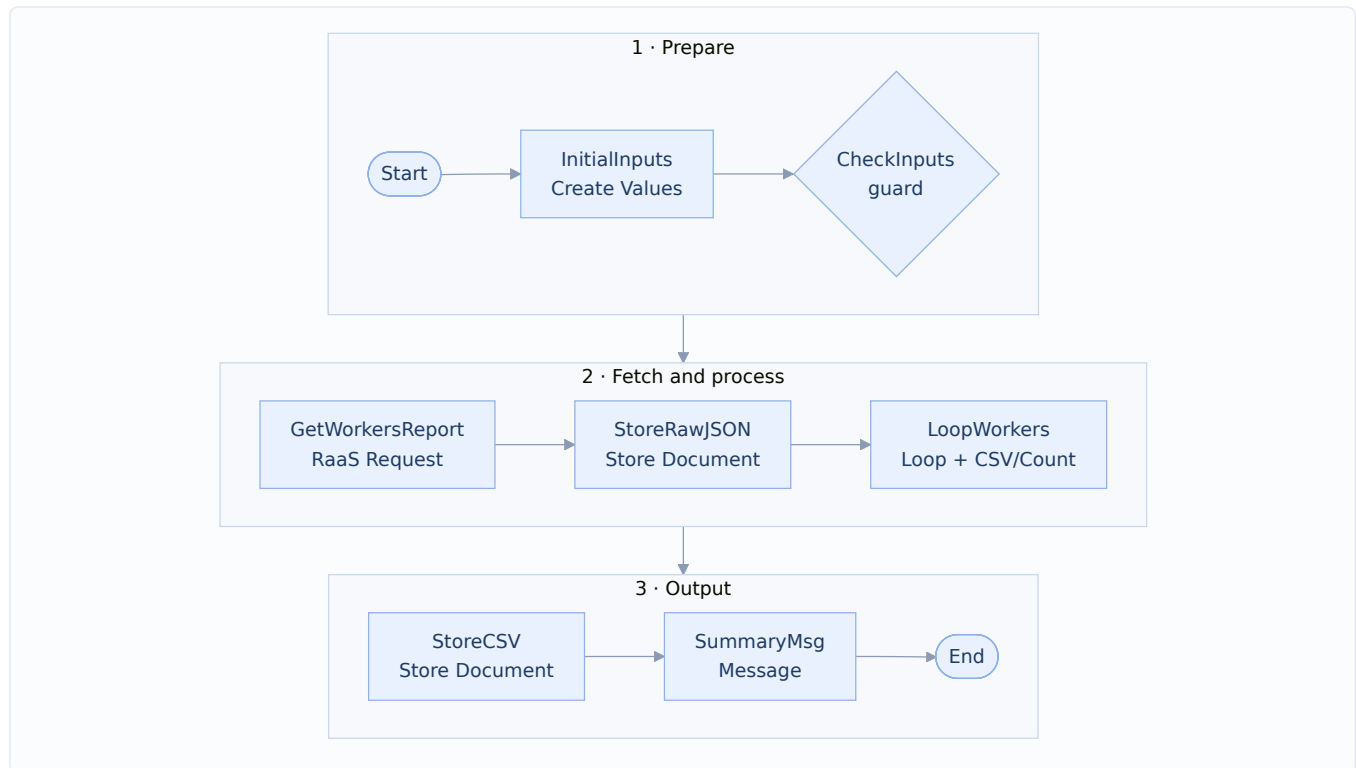
Workday APIs that are not behind the API Gateway (for example some acquired products) count as external calls.

Security checklist for one integration

- ☐ Integration system added to an ISSG with only the domains it needs
- ☐ Get permission on the data it reads; Put / Modify on the data it changes
- ☐ Report shared with the ISSG (RaaS)
- ☐ BP initiate permission if a Put starts a business process
- ☐ External credentials created in orchestration Settings, never typed into steps
- ☐ Security changes activated in each tenant

8. Sample 1 — Call a custom report (RaaS) and produce an output file

Goal: run a worker report filtered by a launch-parameter date, loop through each row, build a CSV output, count the rows, and attach both the raw JSON and the CSV to the integration event.



Tenant preparation

1. Create an advanced custom report, for example `INT_ORCH_Worker_Custom_IDs`, data source *All Active and Terminated Workers* (or *All Workers*), columns Employee ID, Worker, Hire Date, Custom ID. Give each column a clean XML alias (`Employee_ID`, `Worker`, `Hire_Date`, `Custom_ID`).
2. Add a prompt such as `Hired_On_or_After` if you want date filtering.
3. Tick **Enable As Web Service** and share the report with the ISU / ISSG of the integration system.
4. Note the report owner's user name — it is part of the RaaS URL.

Why: column aliases become JSON keys, so short clean aliases give simple JSONPath; sharing gives the ISU access or the call returns nothing.

Build steps

Step 1 — Create Values `InitialInputs`

Key	Expression	Why
<code>fromDate</code>	<code>lp.dateParam("Hired On or After")</code>	Report filter for this run
<code>outFileName</code>	text <code>Workers_</code> + today's date + <code>.csv</code>	Unique file per run

Step 2 — Continue on Conditions `CheckInputs`

Condition `lp.existsAndHasValue("Hired On or After")`; false message: *Launch parameter Hired On or After is required*. **Why:** stops a full-population extract caused by a blank prompt.

Step 3 — Send Workday RaaS Request `GetWorkersReport`

1. URL pill: `<report owner>/INT_ORCH_Worker_Custom_IDs`.
2. Content Type: JSON.

3. Query parameter: `Hired_On_or_After = data.InitialInputs.fromDate (format yyyy-MM-dd)`.
4. Settings: Connect Timeout raised for large reports; retry policy on.
5. Step error handler → Add Integration Message (ERROR, summary *RaaS call failed*) then propagate.

Step 4 — Store Document `StoreRawJSON`

Data = response body of `GetWorkersReport`; file name `RaaSResponse.json`; **attach to integration event = on**. **Why:** keeps an audit copy of what the report returned — invaluable when users ask "why was this worker missing?".

Step 5 — Loop `LoopWorkers`

1. Data Type: JSON iterator (or AutoType).
2. Data Set: iterator over `$.Report_Entry[*]` on the RaaS body.
3. Optional Filter: skip rows where `Employee_ID` is empty.
4. Inside the loop add a Create Values `RowValues` with `empId = Loop.item.stringAtJsonPath("$.Employee_ID")`, `name`, `hireDate`, `customId` (use `...WithDefault` for optional columns).
5. Aggregation 1 — CSV `WorkerCSV`: create a format with header `Employee_ID,Worker,Hire_Date,Custom_ID`, comma separator; map each column to `RowValues`.
6. Aggregation 2 — Count `RowCount`.

Step 6 — Store Document `StoreCSV`

Data = `LoopWorkers.WorkerCSV`; file name = `InitialInputs.outFileName`; attach to event.

Step 7 — Add Integration Message `SummaryMsg`

Severity INFO; Summary `Processed + RowCount + workers`.

Global error handler: Add Integration Message, severity CRITICAL, Summary = `ProcessingError.message`, Detail = `locationPath`.

Test

1. Validate > Save All to App Hub > Promote > Install (first time).
2. Create integration system `INT_ORCH_Worker_Report_Out` (Orchestrate Integration Template), set attributes, add launch parameter *Hired On or After* (Date).
3. Launch with a recent date; check the event for both files and the count message.

WQL alternative

Replace step 3 with Create Text Template (`SELECT employeeID, worker, hireDate FROM allWorkers WHERE hireDate >= '{{InitialInputs.fromDate}}'`) + Send Workday API Request (REST `/data`), and loop over `$.data[*]`. WQL is faster and returns only the fields you ask for; RaaS is better when the logic already lives in a report or uses calculated fields.

9. Sample 2 — Read and validate a CSV file

Goal: pick up a CSV file, parse it into rows, validate each row, and produce a clean-rows count plus an error file. This is the reusable "file reader" pattern that Sample 3 builds on.

Where the file comes from

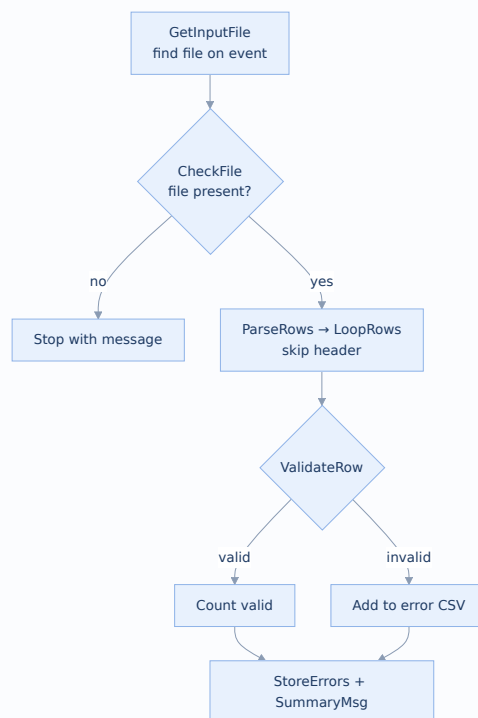
Source	How the orchestration gets it	Use when
Attachment on the integration event	Document functions: filter event documents by label (<code>documents.labels</code>), then <code>get()</code> to return it as Data	Users upload a file when launching, or an upstream integration stores it
SFTP / external endpoint	Retrieval configured on the integration system or a Send HTTP Request to the file API	Files dropped by a third party on a schedule
RaaS with CSV content type	Send Workday RaaS Request, Content Type CSV	Round-trip testing or boomerang flows

For a user-driven load, the simplest is an attachment on the event labelled, for example, `INPUT`.

Sample input file

```
Employee_ID,Custom_ID_Type,Custom_ID,Effective_Date
21001,BADGE_ID,BADGE-0001,2026-10-01
21002,BADGE_ID,BADGE-0002,2026-10-01
21003,BADGE_ID,,2026-10-01
```

Flow



Build steps

Step 1 — Create Values `GetInputFile`

- `inputDoc` = event documents filtered by label `INPUT`, then `get()` (Data).
- `inputText` = `inputDoc` converted to text.

Why: one place that knows where the file lives — switching to SFTP later changes only this step.

Step 2 — Continue on Conditions `CheckFile`

Stop with *No input file attached with label INPUT* when the document is not defined (`isDataDefined`) or the text is empty. **Why:** a missing file should end the run with a clear message, not a null-pointer error mid-loop.

Step 3 — Create Values `ParseRows`

- `rows` = CSV parse/iterator function over `inputText` (search `csv` in Function Explorer), or split `inputText` on the line break.
- `delimiter` = `,` (or read from an integration attribute so pipe files work too).

Why: turns one text blob into items the Loop can iterate.

Step 4 — Loop `LoopRows`

- Data Set = `ParseRows.rows`.
- Filter: exclude the header row (index 0 or first column equal to `Employee_ID`) and blank lines.
- Inside, Create Values `Row`: split the line on the delimiter; `empId` = column 0 trimmed, `idType` = column 1, `customId` = column 2, `effDate` = column 3.

Step 5 — Branch on Conditions `ValidateRow` (inside the loop)

- IF branch *Valid*: `empId` not empty AND `customId` not empty AND `effDate` matches `yyyy-MM-dd`.
- ELSE branch *Invalid*: Add Integration Message (WARNING) *Row skipped: Employee <id> missing Custom ID.*

Why: bad rows are reported, never sent to Workday.

Step 6 — Loop aggregations

- Count `ValidCount` with condition *valid*.
- CSV `ErrorFile` with condition *invalid*, columns `Employee_ID,Reason`.

Step 7 — Store Document `StoreErrors` + Add Integration Message `SummaryMsg`

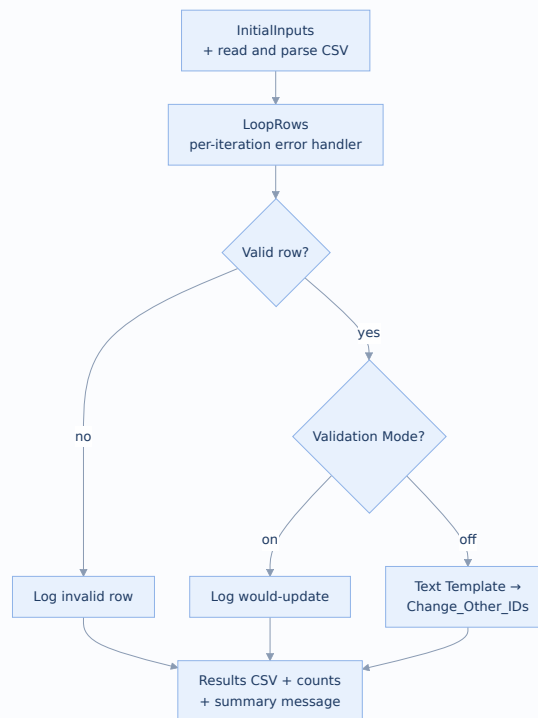
Attach `Errors.csv` to the event only if the error count > 0; summary `Valid rows: N, invalid rows: M`.

Common CSV pitfalls

Issue	Symptom	Fix
Windows line endings (<code>\r\n</code>)	Last column has a hidden <code>\r</code>	Trim every value
UTF-8 BOM	First header reads <code>\uFEFFEmployee_ID</code>	Strip BOM or match header by position
Commas inside quoted values	Columns shift	Use the CSV parse function, not a plain split
Leading zeros (<code>00123</code>)	Lost if file was saved from Excel	Treat IDs as text; agree format with sender
Blank trailing line	Empty row error	Filter blank lines in the loop

10. Sample 3 — Inbound: update worker Custom IDs from an input file

Goal: read the CSV from Sample 2, and for each valid row call the Human_Resources web service **Change_Other_IDs** to add or update the worker's Custom ID. Rows fail individually without stopping the run; a results file shows every row's outcome.



Tenant preparation

1. **Custom ID Type:** confirm it exists (task *Maintain ID Types* / Custom ID types) and note its reference ID, for example `BADGE_ID`.
2. **Integration system:** `INT_ORCH_Worker_CustomID_Inbound`, Orchestrate Integration Template; set Orchestration Name and Application Reference ID.
3. **Launch parameters:** *Validation Mode* (Boolean, default true), *Input File Label* (Text, default `INPUT`).
4. **Integration attribute or map:** default Custom ID Type, in case the file column is blank.
5. **Security:** ISU in an ISSG with Put access to the domain that secures `Change_Other_IDs` (check with *View Security for Securable Item*), Get access to worker data, and initiate rights on the ID-change business process policy if your tenant routes it through a BP. Activate pending security changes.

Why validation mode defaults to true: the first run on any tenant should only report what would change. You flip it to false once the output looks right.

Build steps

Step 1 — Create Values `InitialInputs`

Key	Expression
<code>validateOnly</code>	<code>lp.booleanParam("Validation Mode")</code>
<code>fileLabel</code>	<code>lp.stringParam("Input File Label")</code> , default <code>INPUT</code>
<code>defaultIdType</code>	attribute value or default <code>BADGE_ID</code>
<code>wsVersion</code>	<code>v44.2</code> (or your tenant's current version)

Steps 2-4 — Read, check and parse the file

Exactly as Sample 2 steps 1-3, using `InitialInputs.fileLabel`.

Step 5 — Loop `LoopRows` with Add Error Handler per Iteration

Handler type **Conditional Outputs**; inside it add Add Integration Message (ERROR) with Summary `Row failed for` + `Row.empId` and Detail `ProcessingError.message`. **Why:** one bad Employee ID must not stop the other 999 rows.

Step 6 — Inside the loop: Create Values `Row`

Split columns; `idType` falls back to `InitialInputs.defaultIdType`.

Step 7 — Branch on Conditions `ValidateRow`

Same rules as Sample 2; invalid rows log a WARNING and go to the results file.

Step 8 — Branch on Conditions `CheckMode`

- IF `InitialInputs.validateOnly = true` → Add Integration Message (INFO) *Would set <type> = <id> for <employee>.*
- ELSE → steps 9-10.

Step 9 — Create Text Template `BuildChangeOtherIDs`

```
<bsvc:Change_Other_IDs_Request xmlns:bsvc="urn:com.workday/bsvc" bsvc:version="{{InitialInputs.wsVersion}}">
  <bsvc:Business_Process_Parameters>
    <bsvc:Auto_Complete>true</bsvc:Auto_Complete>
    <bsvc:Run_Now>true</bsvc:Run_Now>
  </bsvc:Business_Process_Parameters>
  <bsvc:Change_Other_IDs_Data>
    <bsvc:Worker_Reference>
      <bsvc:ID bsvc:type="Employee_ID">{{Row.empId}}</bsvc:ID>
    </bsvc:Worker_Reference>
    <bsvc:Custom_Identification_Data bsvc:Replace_All="false">
      <bsvc:Custom_ID bsvc>Delete="false">
        <bsvc:Custom_ID_Data>
          <bsvc:ID>{{Row.customId}}</bsvc:ID>
          <bsvc:ID_Type_Reference>
            <bsvc:ID bsvc:type="Custom_ID_Type_ID">{{Row.idType}}</bsvc:ID>
          </bsvc:ID_Type_Reference>
          <bsvc:Issued_Date>{{Row.effDate}}</bsvc:Issued_Date>
        </bsvc:Custom_ID_Data>
      </bsvc:Custom_ID>
    </bsvc:Custom_Identification_Data>
  </bsvc:Change_Other_IDs_Data>
</bsvc:Change_Other_IDs_Request>
```

Element	Why it is there
<code>Worker_Reference</code> with <code>Employee_ID</code>	Identifies the worker from the file's ID (use <code>Contingent_Worker_ID</code> for contingent workers)
<code>Replace_All="false"</code>	Keeps the worker's other Custom IDs; <code>true</code> would wipe every Custom ID not in the request
<code>ID_Type_Reference</code>	Which Custom ID type to set
<code>Custom_ID_Shared_Reference</code> (optional, not shown)	Needed to update an existing ID instead of adding a second one of the same type — fetch it via RaaS/Get_Workers and add it inside <code>Custom_ID</code>
<code>Issued_Date</code>	Optional; only if your design uses the file date
<code>Auto_Complete</code> / <code>Run_Now</code>	Completes the change without routing for approval

Verify element names against the `Change_Other_IDs` operation for your version in the SOAP API Explorer before go-live.

Step 10 — Send Workday API Request `ChangeOtherIDs`

SOAP, service `Human_Resources`, version = `wsVersion`, body = `BuildChangeOtherIDs`. On success Add Integration Message (INFO) or rely on the aggregation.

Step 11 — Loop aggregations

- CSV `Results` with columns `Employee_ID, Custom_ID_Type, Custom_ID, Status, Message` — status *Updated, Validated, Invalid* or *Failed* (use `isSuccess` from the iteration handler).
- Count `UpdatedCount`, `FailedCount`, `InvalidCount` (each with a condition).

Step 12 — Store Document `StoreResults`

`CustomID_Results.csv`, attached to the event.

Step 13 — Add Integration Message `Summary`

INFO when failed = 0, WARNING otherwise: `Updated X, invalid Y, failed Z`.

Updating an existing Custom ID (optional enhancement)

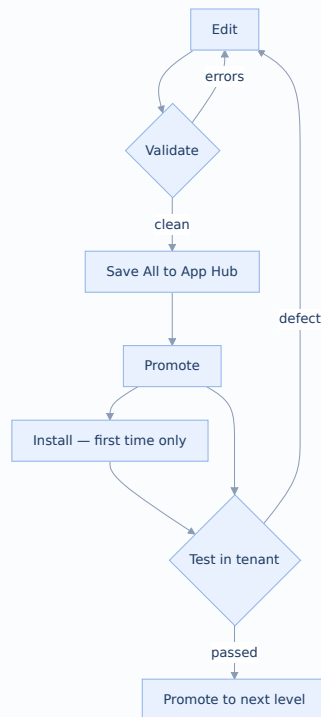
If the worker may already hold an ID of the same type, add a RaaS step before the loop that returns Employee ID, Custom ID Type and the Custom ID's reference, then change `LoopRows` to a **Join Loop** on Employee ID + type. Missing match → add new; single match → include `Custom_ID_Shared_Reference` to update in place. This avoids duplicate IDs of one type.

Test plan

#	Scenario	Expected result
1	Validation Mode = true, 3-row file	No changes; 3 <i>Would set</i> messages; results file
2	Validation Mode = false, valid rows	Custom ID visible on worker profile (Personal > IDs)
3	Unknown Employee ID	Row <i>Failed</i> with Workday's validation message; other rows still load
4	Blank Custom ID	Row <i>Invalid</i> , no web service call
5	No file attached	Run stops at CheckFile with a clear message
6	Re-run same file	No duplicates when the enhancement is in place

11. Deploy, launch, monitor and troubleshoot

Every change follows the same loop: edit → Validate → Save All to App Hub (new version) → Promote → test in the tenant → check the event and Run Logs.



Moving between environments

Environment	Purpose	Note
DEV	Isolated build/test	Extend SKU only
IMPL	Build and structured testing	Usual first test tenant
SBX	UAT on weekly-refreshed production data	Test with real volumes
PROD	Live runs	Promotion is one-way; installed apps cannot be removed

Promotion moves only the app. Move the integration system, launch parameters, attributes, maps, custom reports and security separately (OX 2.0 or manual). Keep a ZIP of each promoted version — there is no automatic rollback, but a ZIP can be re-uploaded.

Launching and scheduling

- Ad hoc: *Launch / Schedule Integration* on the integration system, fill launch parameters.
- Scheduled: same task with a recurrence; use a *Last Successful Run* style date parameter for incremental extracts.
- Chained: Trigger Integration from another orchestration, or an integration event subscription.

Where to look when it fails

Place	Shows	Use for
Integration event > Messages	Add Integration Message entries, platform errors	First stop for functional users
Integration event > Output files	Store Document / Log to File outputs	Results and error files
Run Logs (Builder, bottom panel)	App, Remote, Access, Audit logs	Developer debugging
Build Logs	Save/compile errors across all orchestrations in the app	Save All fails without a validation error

Useful Run Log filters: `wd_env = PROD`, `wd_log_type = remote`, `wd_response_code = 401` or `500`, `wd_message` contains *Validation failed*. Download logs you must keep beyond the retention period.

Common errors

Symptom	Likely cause	Fix
Launch fails immediately, orchestration not found	Typo in Orchestration Name / Application Reference ID, or app not installed	Re-copy from App Overview; check App Manager > Installed
RaaS returns empty or 403	Report not shared with ISU, or not web-service enabled	Share with ISSG; enable as web service
SOAP "Invalid ID value"	Employee ID not found or wrong ID type	Check ID type; log and skip the row
SOAP permission error	Missing domain or BP policy access	View Security for Securable Item; activate changes
Expression cannot find a step output	Step referenced is below the current step	Reorder steps
Paged SOAP call invalid	You added the SOAP envelope yourself	Remove envelope for paged calls
Loop processes header row	No filter on first row	Filter by index or header text
Run stops on first bad row	Default Propagate Error	Add per-iteration handler with Conditional Outputs
Output file not on event	Store Document attach option off	Enable attach to integration event
Launch parameter reads null	Label mismatch	Copy the exact label from the integration system

12. Best practices, build checklist and sources

Best practices

- **Naming:** app by domain (`workerIntegrations`), orchestration by action (`UpdateWorkerCustomID`), steps by purpose (`GetWorkersReport` , not `SendWorkdayRaaSRequest1`).
- **One input step:** gather launch params, attributes and maps in a single Create Values at the top.
- **Fail fast:** Continue on Conditions for required inputs and files before any API call.
- **Validate-only mode** on every inbound integration; default it to true.
- **Row-level resilience:** per-iteration error handler with Conditional Outputs plus a results CSV.
- **Tell the business:** Add Integration Message for start, counts and errors; Log only for developer detail.
- **No hard-coding:** endpoints, ID types and versions in attributes or maps.
- **Reuse:** suborchestrations for lookups, auth and standard logging.
- **Right tool:** RaaS for report logic, WQL for fast targeted reads, paged calls for big SOAP/REST reads; move very large batch loads (hundreds of thousands of rows) to Studio or EIB.
- **Security by least privilege:** only the domains the ISU needs; share reports explicitly.
- **Never paste PII** into online JSONPath testers.

Build checklist

- ☐ App created in the correct company; Short ID suffix visible
- ☐ Orchestration type = Workday Integration System
- ☐ Every step renamed and described
- ☐ Inputs in one Create Values; required params guarded
- ☐ Global error handler logs message and location
- ☐ Loop error handling chosen deliberately (per iteration vs whole loop)
- ☐ Store Document set to attach to integration event
- ☐ Validate clean; Save All to App Hub; Build Logs clean
- ☐ Promoted and installed on target tenant
- ☐ Integration system uses Orchestrate Integration Template; attributes copied exactly
- ☐ Launch parameter labels match orchestration expressions
- ☐ ISSG domains, BP policies and report sharing done and activated
- ☐ Test plan executed in validate-only, then live mode
- ☐ Integration system, report and security migrated alongside the app

Sources

- [Creating Integrations Using Workday Orchestrate — Course Manual](#)
- [Introduction to Workday Orchestrate for Integrations](#)
- [Orchestration Builder Core Concepts](#)
- [Orchestration Data Access](#)
- [Orchestration Logic and Event Flows](#)
- [Getting Started with Integration Apps — Workday Developer Site](#) (sign-in required for full content)

Function names for CSV parsing and document retrieval vary by release — confirm them in the Function Explorer in your tenant's Builder.

Glossary

Term	Simple meaning
Aggregation	A bucket on a loop that collects results (CSV, JSON, count, sum)
App Hub	Where saved app versions live on the Developer Site
Application Reference ID	The ID that tells the integration system which app to run
AutoType	Let Orchestrate decide the value's type
Conditional Outputs	Error-handler mode that lets the flow continue and sets <code>isSuccess</code>
Data Set	The list a Loop walks through

Term	Simple meaning
Integration event	The record of one run: status, messages, files
Integration System (Orchestrate template)	Tenant object that launches an orchestration
ISSG / ISU	Integration security group / integration system user
Iterator	A list of items prepared for looping
JSONPath / XPath	Addresses for finding values inside JSON / XML
Launch parameter	A value typed in when the integration is launched
Pill Mode / Advanced Mode	Visual vs code-style expression editing
Propagate Error	Error-handler mode that passes the error upward
RaaS	A custom report called as a web service
Reference Name	A step's name; how other steps refer to its output
Short ID	Your company's suffix on every app name
Suborchestration	A reusable child orchestration
unwrapSoap	Function that strips the SOAP envelope from a response
WQL	SQL-like query language for Workday data